

ESAP-ER813X-002-SC WISE SDK 编程指南

Elite Semiconductor Microelectronics Technology Inc.

Important Notice

All rights reserved.

No part of this document may be reproduced or duplicated in any form or by any means without the prior permission of ESMT.

The contents contained in this document are believed to be accurate at the time of publication. ESMT assumes no responsibility for any error in this document, and reserves the right to change the products or specification in this document without notice.

The information contained herein is presented only as a guide or examples for the application of our products. No responsibility is assumed by ESMT for any infringement of patents, copyrights, or other intellectual property rights of third parties which may result from its use. No license, either express, implied or otherwise, is granted under any patents, copyrights or other intellectual property rights of ESMT or others.

Any semiconductor devices may have inherently a certain rate of failure. To minimize risks associated with customer's application, adequate design and operating safeguards against injury, damage, or loss from such failure, should be provided by the customer when making application designs.

ESMT's products are not authorized for use in critical applications such as, but not limited to, life support devices or system, where failure or abnormal operation may directly affect human lives or cause physical injury or property damage. If products described here are to be used for such kinds of application, purchaser must do its own quality assurance testing appropriate to such applications.

Revision History

Rev	Date	Description of Change	Approved
1.0	2026.07.16	First Release	
1.1	2026.07.22	文书上的错别字修正 标题的修正 排版修正	

章节目录

1	基本介绍	7
1.1	支援平台	8
1.2	储存器结构	10
1.2.1	启动流程	11
2	WISE 代码包概述	13
2.1	层级结构	13
2.1.1	PHY (物理层)	13
2.1.2	HAL (platform-intf, 硬件抽象层)	14
2.1.3	Core (api, 核心驱动层)	14
2.1.4	Middleware (wise_function, 中间层)	14
2.1.5	Network Protocol (网络协议层)	14
2.1.6	Application(应用层)	15
2.2	目录结构	15
2.3	应用层示例项目	15
2.3.1	Project_template	16
2.3.2	AppLoader	16
2.3.3	WISEDemoApp	18
3	入门指南	22
3.1	CLI 入门指南 (AppLoader)	22
3.1.1	联机设定	22
3.1.2	进入 Console 模式	22
3.1.3	列出支持指令	22
3.1.4	输入 fs info 确认当前储存器分区	23
3.1.5	启动下载 APP 韧体	23
3.1.6	再确认分区信息 (非必要)	24
3.2	Radio TRX 入门指南 (WISEDemoApp)	26
3.2.1	功能概述	26
3.2.2	运作流程	26
3.2.3	RX 事件处理(rfEventHandler)	27

3.2.4 Shell 指令	27
3.2.5 Shell 测试 – 启动讯息	28
3.2.6 Shell 测试 – 启动 Tx/Rx	28
4 应用层接口界面	30
4.1 入口页面摘要	30
4.2 函数摘要	31

图表目录

图 1. WISE SDK 开发包	7
图 2. 芯片模块正反面图	8
图 3. 模块使用示意图	8
图 4. 芯片中的储存器 (白色方块)	10
图 5. 储存器启动顺序 (左: 无内建二阶启动时, 右: 有内建二阶启动时)	11
图 6. 代码结构图	13
图 7. Apploader 在 SBL 中的运作流程	18
图 8. 在 WISEDemoApp 中如何选择示例	19
图 9. 如何在 TeraTerm 上启动 Kermit 传输	25
图 10. kermit 传输中的面板	25
图 11. 芯片模块的对接测试 (左: 装置 A, 右: 装置 B)	29
图 12. 应用层接口说明入口	30

表格 1. 芯片模块引脚描述 (J16)	9
表格 2. 芯片模块引脚描述 (J17)	9
表格 3. 芯片的存储器功能说明	11
表格 4. 现有应用层示例摘要	16
表格 5. 存储器于 Apploader 中的默认分区配置 (512KB)	17
表格 6. WISEDemoApp 中的可选示例	19
表格 7. Topics 页面摘要	30
表格 8. Data Structure 页面摘要	30
表格 9. Files 页面摘要	31

表格 10. Topics 的函数说明摘要.....	31
----------------------------	----

Confidential

1 基本介绍

WISE SDK 是用于开发 ESMT SoC 芯片-ER8130(之后都”芯片”或”SoC”称呼)的整合性软件开发工具包，主要包含开发环境(编程环境)，示例代码包，以及图形界面的刻录工具。此章节大略的说明 SDK 的内容以及硬件方面的信息，让开发者可较易进入状况，其详细数据应参考相关文件。

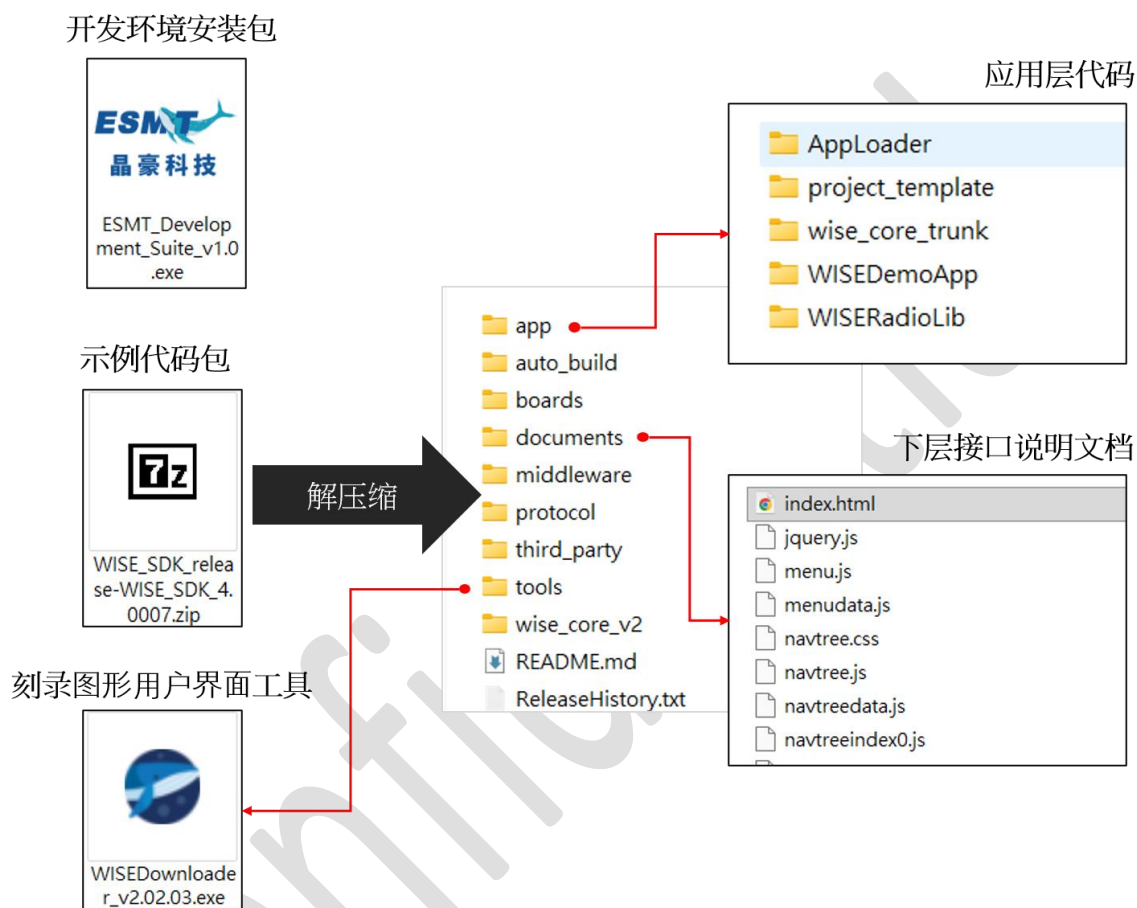


图 1. WISE SDK 开发包

- 开发环境
 - ✓ 安装档形式，安装后会建立一个基于 Eclipse 的编程编译环境
- 示例代码：
 - ✓ 提供芯片全功能的 driver API，供应用程序开发使用
 - ✓ 基于 driver API 实作多种 中间层，降低开发工作量
 - ✓ 以 shell command 形式提供完整功能演示
- 刻录工具
 - ✓ 图形界面，搭配 DAPLink 以 SWD 界面刻录 ER8130，亦可操作清除及比对

1.1 支援平台

本章节将大略地说明 WISE SDK 所支持的 ER8130 芯片模块如图 2，详细信息请参考芯片硬件相关文件。引脚说明如表 1 及表 2。要提供芯片电源可透过两个路径。

- 将排针 PWR-C 短路，并接上 USB，VDDH 便可由板上 LDO 取得 3V
- 将排针 PWR-C 开路，便可外部供给 3V（建议范围为 1.8~3.6V）至 PWR-A 或 PWR-C 的 VDDH

若是同时将 PWR-C 以及 IO-UART 短路，那么便可透过 USB 接上计算机看到虚拟串口。

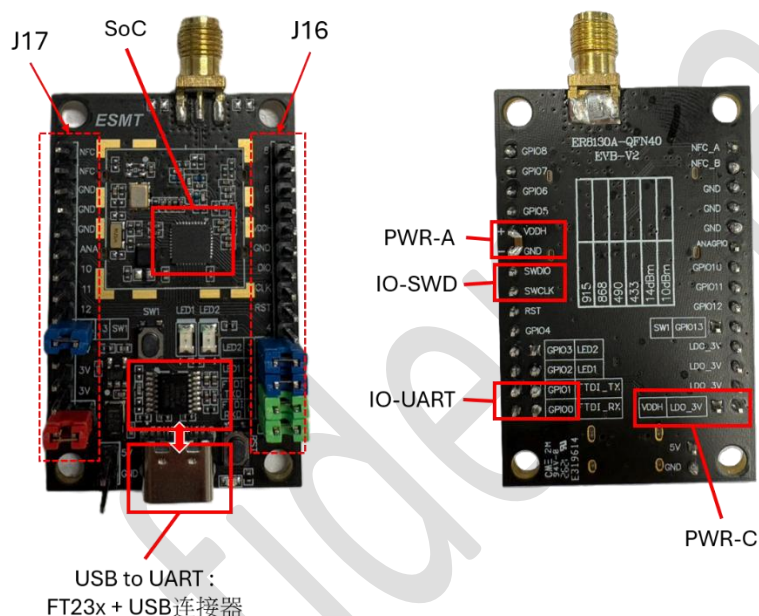


图 2. 芯片模块正反面图

下图为模块与计算机端连接的关系示意图。

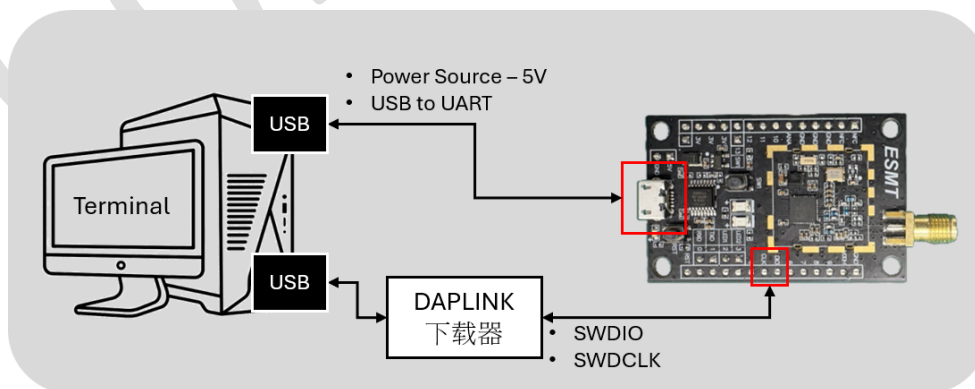


图 3. 模块使用示意图

表 1, 表 2 提供引脚描述, 以利开发者能快速进入状况。注意, 在”芯片引脚”后面括号所标注的引脚编号是 QFN40 包装下的芯片引脚脚位。

表格 1. 芯片模块引脚描述 (J16)

功能	J16 引脚名		功能描述
	芯片引脚(引脚编号)	可绑定引脚	
IO	GPI05 (33)		芯片 通用引脚 5
	GPI06 (34)		芯片 通用引脚 6
	GPI07 (35)		芯片 通用引脚 7
	GPI08 (36)		芯片 通用引脚 8
PWR-A	VDDH		芯片 工作电压输入端
	GND		芯片 接地端
SWD	SWDIO (32)		芯片 SWD 资料
	SWCLK (30)		芯片 SWD 频率
	RST (24)		芯片 复位输入 (低位触发)
IO	GPI04 (29)		芯片 通用引脚 4
IO-LED	GPI03 (28)	LED2	芯片 通用引脚 3, 板上短路可与 LED2 绑定
	GPI02 (27)	LED1	芯片 通用引脚 2, 板上短路可与 LED1 绑定
IO-UART	GPI01 (26)	FTDI_TX	芯片 通用引脚 1/UART_RXD, 板上短路可与 FT23x 绑定
	GPI00 (25)	FTDI_RX	芯片 通用引脚 0/UART_TXD, 板上短路可与 FT23x 绑定

表格 2. 芯片模块引脚描述 (J17)

功能	J17 引脚名		功能描述
	芯片引脚(引脚编号)	可绑定引脚	
NFC ANT	NFC_A (6)		芯片 NFC 天线
	NFC_B (5)		芯片 NFC 天线
PWR-B	GND		芯片 接地端

ADC	ANAGPIO (12)		芯片 ADC 输入引脚
IO	GPI010 (20)		芯片 通用引脚 10
IO	GPI011 (21)		芯片 通用引脚 11
IO	GPI012 (22)		芯片 通用引脚 12
IO-SW	GPI013 (23)	SW1	芯片 通用引脚 13，板上短路可与 SW1 绑定
		LDO_3V * 3	板上 LDO 输出 (3V)，输入电压源来自 USB 5V
PWR-C	VDDH	LDO_3V	芯片 工作电压输入端，板上短路可直接取用 LDO 3V

1.2 储存器结构

下图为芯片系统内的储存器配置。系统由 ARM Cortex-M3 核心透过内部总线与各周边及存储器互连，芯片上电后控制流必定先进入 Boot ROM，执行固化启动代码，再依 eFuse 中的旗标设定决定后续行为，最终透过 XIP (execute-in-Place，就地执行) 接口跳转至 Flash 执行使用者韧体。

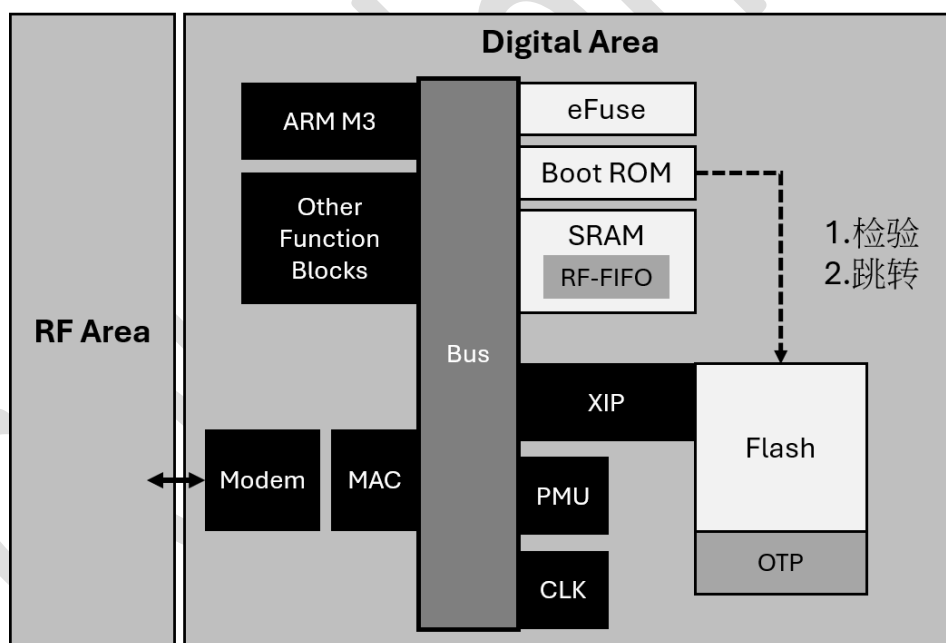


图 4. 芯片中的储存器（白色方块）

各储存器说明如下表：

表格 3. 芯片的存储器功能说明

储存器	大小	描述
eFuse	1 KB	一次性可程序化内存，存放系统永久性旗标，包含安全启动密钥（SHA Key）及 Boot Mode 控制旗标；写入后不可复原
Boot ROM	16 KB	芯片出厂时已固化的启动代码存放处；芯片上电后必定先进入此处，依 eFuse 旗标决定执行安全验证、停留等待刻录，或直接跳转至 Flash
Flash	512 KB	透过 XIP 接口提供就地执行；存放用户韧体（SBL、应用程序）及 Flash 分区数据
OTP	512B*3	3 个 512 个字节的一次性写入区域，存放固件重要数据，如 CRC
SRAM	64 KB	执行期数据存放区，包含全局变量、堆栈（Stack）、Heap，以及 RF FIFO 缓冲区

1.2.1 启动流程

芯片支持两种启动架构，如图 5 所示：

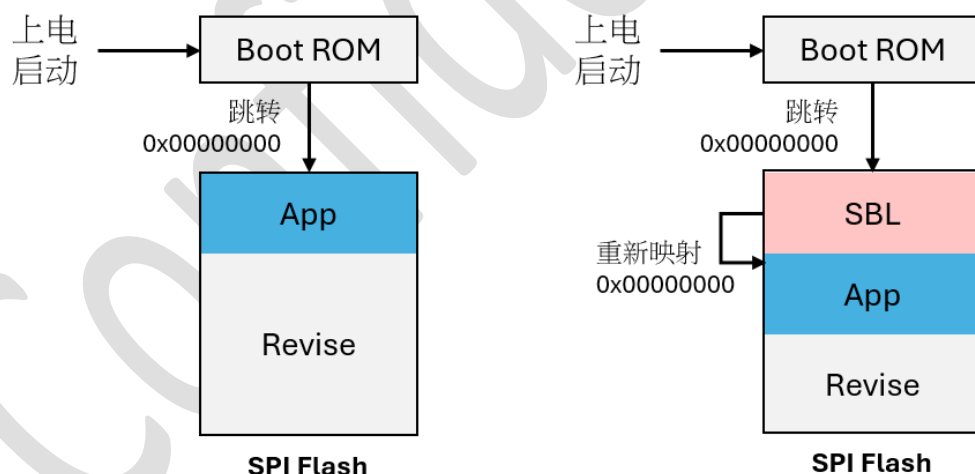


图 5. 储存器启动顺序(左:无内建二阶启动时，右:有内建二阶启动时)

- **无二阶启动（左）**：Boot ROM 初始化后直接跳转至 Flash 地址 0x00000000，0 由用户应用程序接管执行。适合不需要固件空中更新（OTA/IAP）的简单应用。
- **有二阶启动（右）**：先在地址 0 烧入引导启动程序(Bootloader)，Boot ROM 跳转至 Flash 0x00000000，由 SBL 接管。SBL 完成分区验证后透过地址重新映像（Remap）将控制权交给应用程序。此架构支持 OTA 及 UART 刻录（IAP），为 WISE SDK 的预设使用方式。

**** SBL (Second Bootloader) 中的引导启动程序，在 WISE SDK 有提供示例代码，如 AppLoader (有档案分区验证系统) 或是 WISEDemoApp\demo_boot_loader (简单刻录及跳转)**

2 WISE 代码包概述

本章说明 WISE SDK 解压缩后的目录配置、各软件层级的职责划分，以及目前 SDK 内建可供参考、移植的应用层示例项目。

2.1 层级结构

WISE SDK 采用分层式架构设计，由下而上分别为

PHY → HAL → Core → Middleware / Network Protocol → Application，

各层职责如下图：

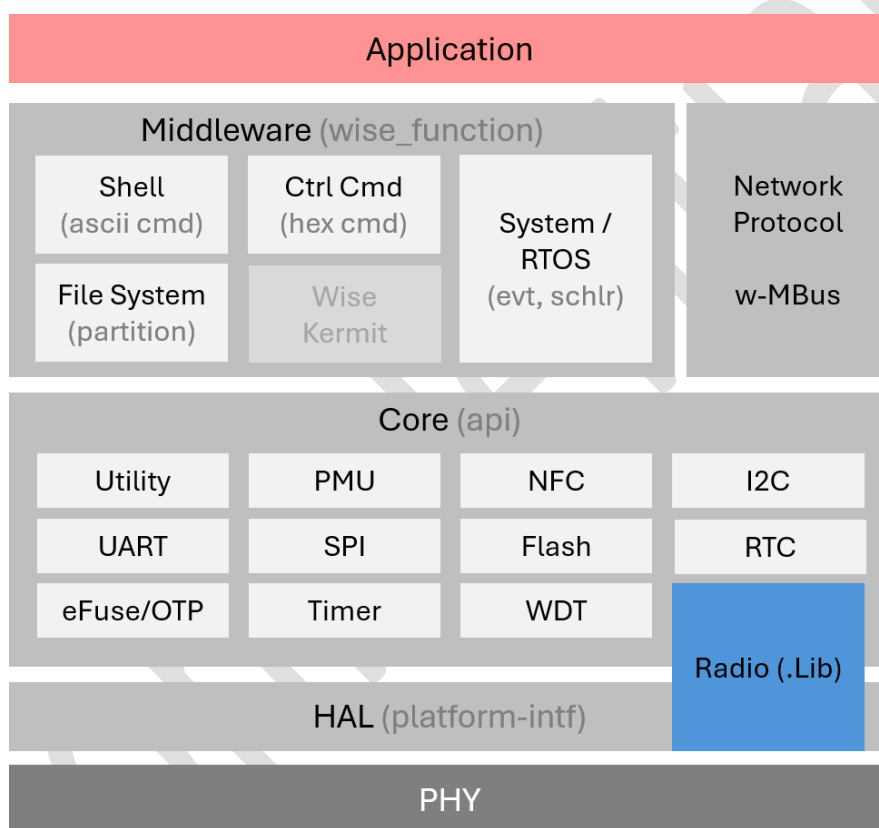


图 6. 代码结构图

2.1.1 PHY (物理层)

对应芯片本身的模拟与射频硬件电路，是整个软件堆栈最底层的硬件基础。

2.1.2 HAL (platform-intf, 硬件抽象层)

封装与特定芯片型号相关的缓存器存取、中断向量等平台相依细节，让上层的 Core API 不需因应不同芯片而改动，对应原始码目录为 wise_core_v2/platform/。

2.1.3 Core (api, 核心驱动层)

提供完整的周边 driver API 供应用程序与 Middleware 呼叫，分为两种型态：

开放原始码周边 driver(白色方块)：

Utility / PMU / NFC / I2C / UART / SPI / Flash / RTC / eFuse-OTP / Timer / WDT,

对应 wise_core_v2/api/ 下逐一实作，经由 HAL 存取硬件。

Radio(.Lib) (蓝色方块)：

RF 收发物理层，以预编译函式库型式提供

wise_core_v2/prebuilt_libs/libWISERadioLib_****.a，应用程序透过以下 api 头文件调用：

wise_radio_api / wise_radio_802154_api / wise_radio_ble_api / wise_radio_wmbus_api

2.1.4 Middleware (wise_function, 中间层)

建立在 Core API 之上，提供通用的软件服务以降低应用程序开发复杂度，对应原始码目录为 middleware/：

Shell(ascii cmd):文字交互式命令行接口，供开发者以终端机软件(如 TeraTerm)输入可读指令进行除错与功能示范。

Ctrl Cmd(hex cmd):二进制封包协议(preamble + length + payload + CRC8)，由 host 端主动发起指令、装置端被动响应，用于量产治具或上位机对装置进行远程控制(缓存器读写、文件系统刻录等)。

System / RTOS(evt, schlr):裸机环境下的简易事件与排程机制，在导入 FreeRTOS 的项目中亦身兼 FreeRTOS 移植组态层(提供 FreeRTOSConfig.h)。

File System(partition):Flash 分区管理，提供韧体与用户数据的分区读写、格式化与验证。

Wise Kermit:透过 Kermit 通讯协议进行韧体文件传输刻录，主要用于 AppLoader 开机程序中人工以终端机软件传送应用程序映像。

2.1.5 Network Protocol (网络协议层)

与 Middleware 同层但独立分类，目前包含 w-MBus 协议堆栈(数据链结层原始码 加上 PHY 函式库)，原始码实际上位于独立的 protocol/ 目录而非 middleware/，定位为「特定应用的无线通信协议」，与通用中间件服务区隔。

2.1.6 Application(应用层)

用户最终的产品逻辑所在，呼叫 Middleware 与 Core API 完成产品功能，SDK 内建的 AppLoader、WISEDemoApp、project_template 等专案皆属此层。

2.2 目录结构

WISE SDK 解压缩后的顶层目录配置如下，app/ 底下为可直接汇入 IDE 建置的项目，middleware/、wise_core_v2/ 为底层共享的链接库，依上一节所述的层级架构划分：

wise_sdk/	顶层目录
— app/	项目目录
— AppLoader/	应用程序加载器 (bootloader)
— project_template/	含 UART 功能的简易 Hello World 项目
— WISEDemoApp/	整合型示范应用程序
— boards/	各开发板组态设定
— documents/	文件
— middleware/	软件功能模块
— wise_ctrl_cmd/	二进制控制指令 (host 端远程控制)
— wise_flash_filesystem/	Flash 分区管理
— wise_shell_v*/	Console 命令行界面
— wise_system/	裸机环境下的 RTOS-like API
— wise_kermit/	Kermit 通讯协议实作
— wise_timer_hub/	单一 WUT 硬件上的多信道软件定时器服务
— retarget/	stdio 的包装层
— protocol/	网络协议层
— wmbus_dataink/	w-MBus 数据链结层原始码 + PHY 预编译函式库
— wise_core_v2/	ESMT 芯片 ER 系列核心层
— api/	应用程序开发共享 WISE API
— platform/	ESMT 芯片 平台 HAL driver
— prebuilt_libs/	预编译函式库 (如 Radio 收发函式库)

2.3 应用层示例项目

WISE SDK 内建三个可直接汇入 IDE 的应用层项目:AppLoader、project_template、WISEDemoApp，其中 WISEDemoApp 为整合型示范代码，依功能分为「流程实作」「内部区块」「数位接口」「无线通信」四大类，可在 IDE 中切换启用其中任一示例。详细列表如下。

表格 4. 现有应用层示例摘要

专案名	描述
AppLoader	包含完整文件系统的引导加载程序，实现存储器分区的建立，档案刻录以及错误验证。使用 UART (GPIO0, 1) 来执行指令及数据传输
Project_template	实现 UART “Hello world” 的空白专案，用户若要新建应用流程可以此开始
WISEDemoApp	整合型的示例代码，可在 IDE 中启用其中各种周边及内部区块的示例

2.3.1 Project_template

最小化的起始专案，仅初始化 UART(GPIO0/1, 115200 8N1)并输出 “Hello World”。适合作为客户自定义应用的空白起点，不含 Shell 或 Middleware 依赖，编译后即可直接刻录验证环境是否正常。

2.3.2 AppLoader

此为完整功能的两阶段启动架构中的第二阶段启动程序(2nd Bootloader, SBL)，除了提供”在应用编程(IAP)”，同时建立储存器的文件系统，即分割判断存储器分区(partition)，并且在刻录及开机时判断分区。

上电后由 ROM 跳转执行，负责以下工作：

- Flash 分区管理（建立/查询分区表）
- Kermit 协议韧体接收
- 验证并跳转至应用程序

AppLoader 运作时的对外通讯接口为 UART，其中 UART_TX 对应 IO0，UART_RX 对应 IO1。支持的互动模式有两种：CLI（命令行接口，供开发人员于终端机手动输入指令）以及 wise_ctrl_cmd（结构化控制命令，供 Host 端工具程序自动化操作）

2.3.2.1 Flash 分区配置

表格 5 为 512KB Flash 下 AppLoader 的默认分区配置，各分区的起始地址与大小由 wise_flash_filesystem 模块在执行 fs format 时自动建立。其中 Free 分区的大小会依 Flash 实际容量自动裁切，FS Table 固定占用 Flash 末端 1 个最小抹除单位（sector, 4KB）。但各分区大小可利用工具透过 wise_ctrl_cmd 调整。

表格 5. 存储器于 Apploader 中的默认分区配置 (512KB)

分区		Default Address	
名称	定义	Offset Address	Size (KB)
SBL	第二阶段引导块	0x00000000	64
AP	应用固件区	0x00010000	192
FU Temp	固件更新暂存区	0x00040000	192
Free	用户自定义数据	0x00070000	56
FS Table	文件系统表	0x0007F000	4

- SBL: 第二阶段引导块, 存放 AppLoader 本身, 建议固定长度, 最小 16 KB, 但实际大小需参考 AppLoader 本身编译出来的大小, 而 AP 也会配合向后移动, 但除 SBL 区块外的起始及结束地址需在代码内修改, 或是透过 wise_ctrl_cmd 修改
- AP: 应用固件区, 存放 WISEDemoApp 等应用程序, 起始地址固定在 SBL 后的第一个有效 Sector。
- FU Temp: 固件更新暂存区, 接收新韧体时暂存于此, 更新完成后由 SBL 复制至 AP 区; 客户可暂用此区, 但 FW update 流程会清除其内容。
- Free: 用户自定义数据区, 可供应用程序存放用户数据, 容量依 Flash 实际大小自动决定。
- FS Table: 文件系统表, 位于 Flash 末端, 占 1 个 sector (4 KB), 储存分区表 header 及各分区的 offset / length 信息。

2.3.2.2 开机流程

图 7 说明 AppLoader 在 SBL 中的完整运作流程。上电后 Boot ROM 跳转至 SBL (AppLoader) 执行, 依序判断:

- (01) 是否强制进入 UART 模式 (如按键触发)? 若是, 直接进入 UART console 等待指令。
- (02) FS Table 是否存在? 若否 (尚未执行 fs format), 进入 UART 模式。
- (03) FU Temp 区是否有待更新韧体? 若有, 将 FU → AP 复制后重启。
- (04) AP 区韧体是否有效? 若有效, 跳转至应用程序执行; 若无效, 回到 UART 模式等待刻录。

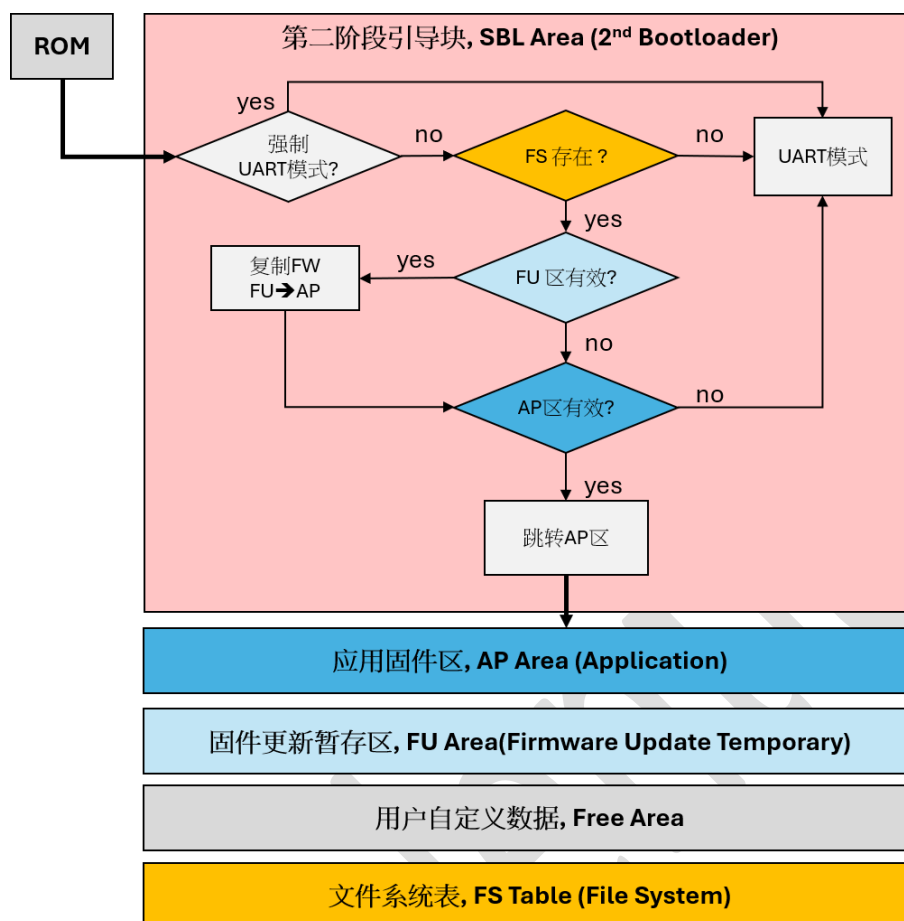


图 7. Apploader 在 SBL 中的运作流程

2.3.3 WISEDemoApp

整合型示范应用，包含 25+ 个功能示例，可在 Eclipse 中透过对项目按下鼠标右键叫出选单，并在选单上如下选择主题即可切换演示代码。

Build Configuration → Set Active → *-demo_XXXXX

--

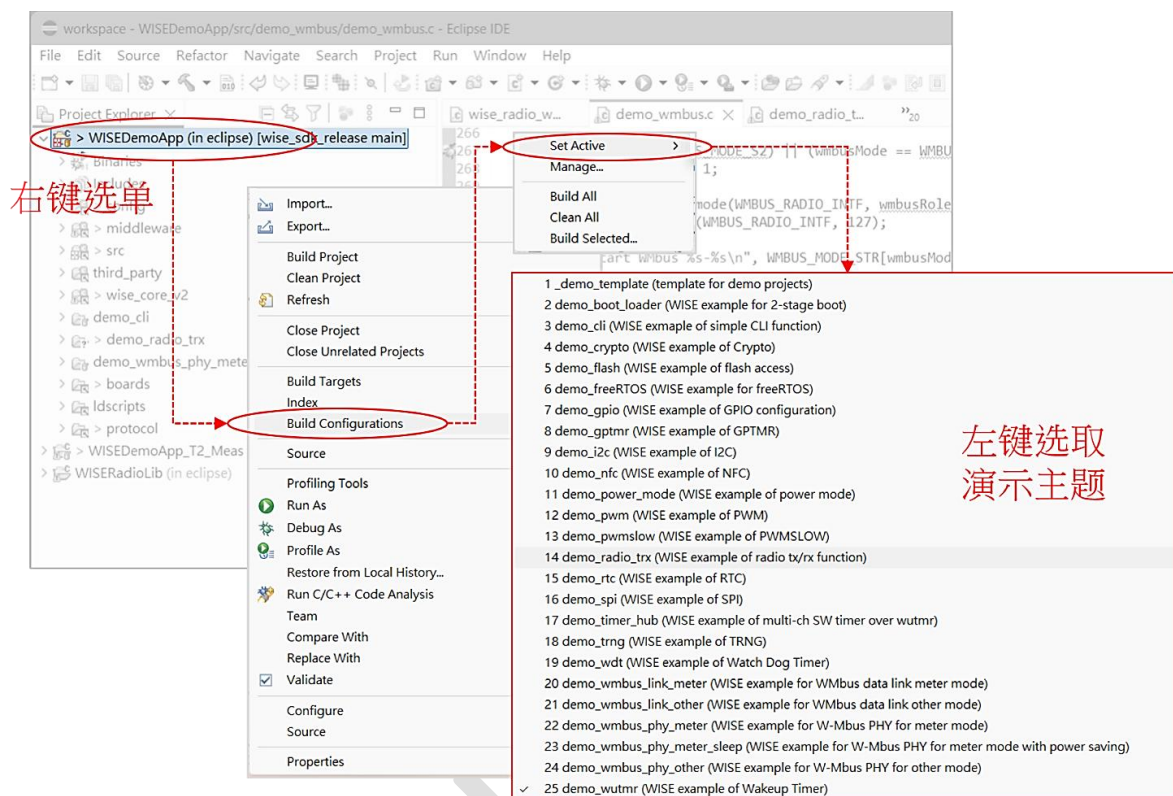


图 8. 在 WISEDemoApp 中如何选择示例

上电后透过 Shell 接口操作，各示例的指令说明请参考第 3 章入门指南及对应 demo_xxx.c 原始码。WISEDemoApp 内可选示例如表格 6。

表格 6. WISEDemoApp 中的可选示例

类型	示例名称	示例描述
流程 实作	_demo_template	空白演示模板 app\WISEDemoApp\src\demo_template\demo_tempalte.c
	demo_boot_loader	简易的加载程序，只包含 Kernit 刻录及指定地址跳转 app\WISEDemoApp\src\demo_boot_loader\demo_boot_loader.c
	demo_cli	简易的命令行实作 app\WISEDemoApp\src\demo_cli\demo_cli_main.c
	demo_freertos	freeRTOS app\WISEDemoApp\src\demo_freertos\demo_freertos_main.c
内部	demo_power_mode	电源模式切换

区块		app\WISEDemoApp\src\demo_power_mode\demo_power_mode.c
	demo_wutmr	唤醒定时器 (WUT) app\WISEDemoApp\src\demo_wutmr\demo_wutmr_main.c
	demo_pwm	PWM app\WISEDemoApp\src\demo_pwm\demo_pwm_main.c
	demo_pwmslow	低速 PWM (16KHz) app\WISEDemoApp\src\demo_pwmslow\demo_pwmslow_main.c
	demo_gptmr	通用定时器 app\WISEDemoApp\src\demo_gptmr\demo_gptmr_main.c
	demo_crypto	加解密操作 app\WISEDemoApp\src\demo_crypto\demo_crypto_main.c
	demo_flash	存储器读写 app\WISEDemoApp\src\demo_flash\demo_flash.c
	demo_timer_hub	软件实作多信道切换 WUT 操作 app\WISEDemoApp\src\demo_timer_hub\demo_timer_hub.c
	demo_trng	随机数生成器 app\WISEDemoApp\src\demo_trng\demo_trng_main.c
	demo_wdt	看门狗定时器 app\WISEDemoApp\src\demo_wdt\demo_wdt_main.c
	demo_rtc	实时时钟 app\WISEDemoApp\src\demo_rtc\demo_rtc_main.c
数位 接口	demo_i2c	I2C app\WISEDemoApp\src\demo_i2c\demo_i2c_main.c
	demo_spi	SPI app\WISEDemoApp\src\demo_spi\demo_spi_main.c
	demo_gpio	GPIO 配置 app\WISEDemoApp\src\demo_gpio\demo_gpio_main.c
无线 通信	demo_radio_trx	radio tx/rx 操作 app\WISEDemoApp\src\demo_radio_trx\demo_radio_trx_main.c
	demo_wmbus_link_meter	WMBus data link meter mode app\WISEDemoApp\src\demo_wmbus_link\demo_wmbus_link.c
	demo_wmbus_link_other	WMBus data link other mode

		app\WISEDemoApp\src\demo_wmbus_link\demo_wmbus_link.c
	demo_wmbus_phy_meter	W-Mbus PHY for meter mode app\WISEDemoApp\src\demo_wmbus\demo_wmbus.c
	demo_wmbus_phy_other	W-Mbus PHY for other mode app\WISEDemoApp\src\demo_wmbus\demo_wmbus.c
	demo_nfc	NFC 操作 app\WISEDemoApp\src\demo_nfc\demo_nfc.c

3 入门指南

3.1 CLI 入门指南(AppLoader)

先前在 2.3.2 开头时提过 AppLoader 提供两种互动方式，即 CLI(命令行接口)及 wise_ctrl_cmd，使用 CLI 建立分区，刻录文件的步骤如下：

3.1.1 联机设定

芯片的 UART 脚位如下

- UART Tx → IO 0
- UART Rx → IO 1

在计算机端终端器上设定如下

- 115200 8N1

3.1.2 进入 Console 模式

按下模块上的 reset 按钮触发板子重置

开机讯息范例：

```
ESMT SBL v2.00 running @00000701
Press c or to start console mode
```

开机后 3 秒内按下 'cccc'加上 enter 中断开机流程，进入 console 模式

```
ESMT Sphynx APP Loader V2.02
Press 'cccc' to enter console...
.cccc
unknown command
ESMT>
```

3.1.3 列出支持指令

输入 help 后可列出支持指令如下：

```
ESMT> help
Usage: help [subcommand]

  help      help
  fs        fs cmds
  kermit    start kermit receiver
  reset     do chip reset
  dump      dump buffer
```

详细说明如下：

Help	列出所有支持指令
reset	触发 CPU 软件重置
fs	文件系统相关指令
└─ fs info	列出分区信息 (若文件系统有效)
└─ fs format	建立预设分区表
Kermit	透过 kermit 文件传输下载应用程序至 flash
└─ kermit	[fs/flash/ram] [partition/flashAddr/ramAddr]

3.1.4 输入 fs info 确认当前储存器分区

```
ESMT> fs info
ESMT file system not found//表示无任何分区
```

3.1.5 启动下载 APP 韧体

下载前先建立文件系统

下达以下命令后, AppLoader 会建立如 2.3.2.1 中所描述的档案分区

```
ESMT> fs format
```

建立完成后回报如下讯息:

```
Create 4 partitions...
Finish
Initialize partions...
Par 0 00000000-00010000
Par 1 00010000-00030000
Par 2 00040000-00030000
Par 3 00070000-0000e000
```

可由讯息中了解如下:

Par 0 (SBL) 为 起始地址 0x00000000, 大小 0x00010000 (64KB),
Par 1 (APP) 为 起始地址 0x00010000, 大小 0x00030000 (192KB),
Par 2 (FU) 为 起始地址 0x00040000, 大小 0x00030000 (192KB),
Par 3 (FREE) 为 起始地址 0x00070000, 大小 0x0000E000 (56KB),

3.1.6 再确认分区信息 (非必要)

此时再透过 fs info 读取分区信息时就不会再读到 ESMT file system not found。

```
ESMT> fs info
FS Info
  ver: 1
  attr: FF
  crc: AC4F
  parNum: 4
  partition 0 00000000-00010000
    sig: ESBL
    dataLen: ffffffff
  partition 1 00010000-00030000
    sig: ESAP
    dataLen: ffffffff
  partition 2 00040000-00030000
    sig: ESFU
    dataLen: ffffffff
  partition 3 00070000-0000e000
    sig: ESUR
    dataLen: ffffffff
```

启动 kermit 接收端

```
ESMT> kermit fs 1
```

在终端机软件 (如 TeraTerm) 启动 kermit 传送(如图 9 及图 10)，那么可以在

“File -> Transfer -> Kermit -> Send” 找到，接着选择二进制文件，路径如下：

app\WISEDemoApp\eclipse\demo_radio_trx\WISEDemoApp_demo_radio_trx.bin

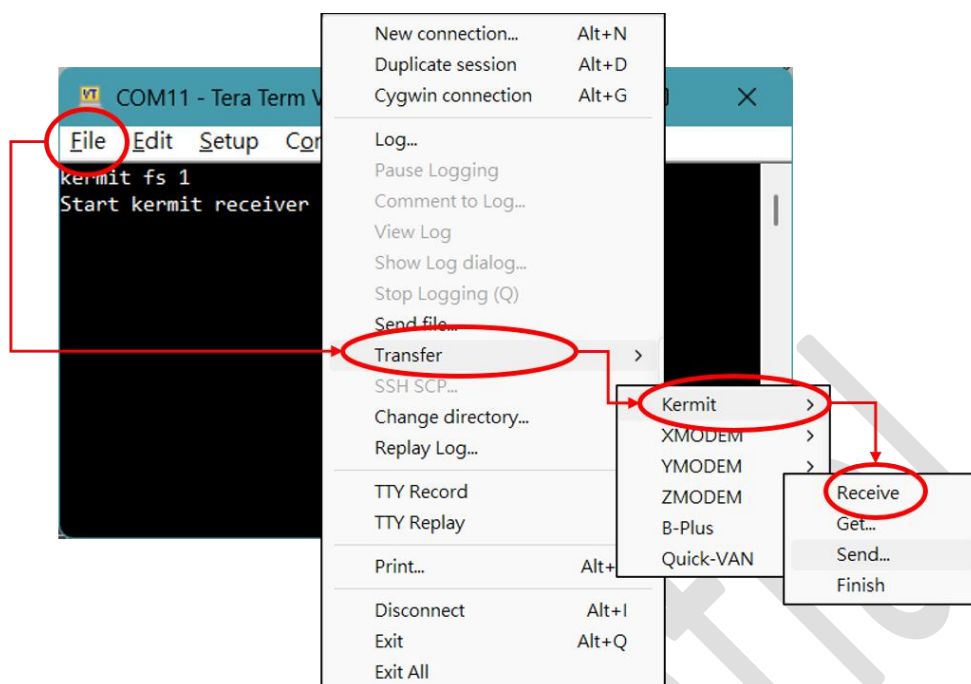


图 9. 如何在 TeraTerm 上启动 Kermit 传输

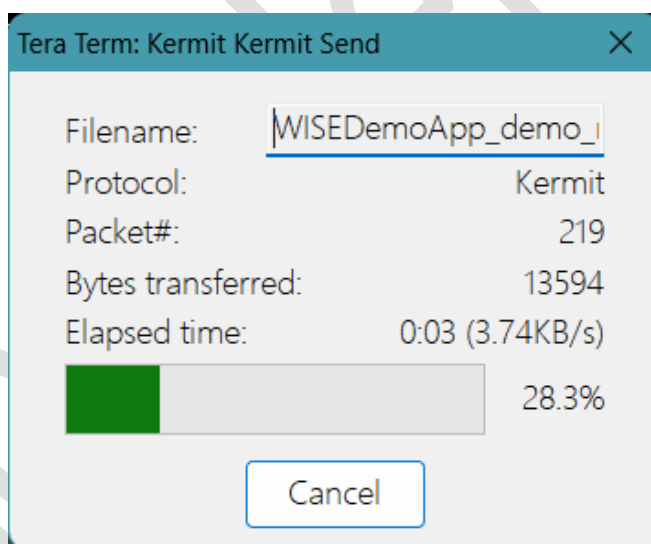


图 10. kermit 传输中的面板

下载完成后重置系统，进入正常开机流程

3.2 Radio TRX 入门指南 (WISEDemoApp)

此章节以 WiseDemoApp\demo_radio_trx 为例，说明要如何开始着手。在开发测试前，先透过 UART 接上芯片 EVB 与计算机，接着便能透过 UART Shell 控制芯片。

3.2.1 功能概述

透过 UART Shell 控制无线电收发，而示例将示范 WISE Radio API 的基本使用流程：

- 无线电 PHY 参数设定
- 封包格式设定
- TX 发送固定测试封包
- RX 接收并印出封包内容
- 频道查询与切换

3.2.2 运作流程

在 demo_radio_trx_main.c 的代码入口 (void main(void)) 中，其实际流程如下：

```
demo_app_common_init()           // 系统 + UART + Shell 初始化
    ↓
wise_radio_init(intf=0)           // 初始化无线电接口 0
wise_radio_set_evt_callback()     // 注册事件 callback → rfEventHandler
wise_radio_set_buffer()           // 提供 512 byte RX 缓冲池
    ↓
wise_radio_config()               // 套用 PHY 设定 + 封包格式
wise_radio_set_tx_pwr_dbm(11)     // TX 功率 11 dBm
wise_radio_set_tx_io_mode(BLOCKING) // TX 为阻塞模式
wise_radio_set_rx_mode(CONTINUOUS) // RX 为连续接收模式
    ↓
while(1) wise_main_proc()         // 主循环，驱动 Shell + 背景任务
```

在 wise_radio_config() 中将设置 PHY 层的 RF 特性及定义封包格式。

RF 特性如下：

- 调变方式 : GFSK
- 频率范围 : 915 MHz~935 MHz
- 频道间距 : 1 MHz (共 21 个频道, ch0~ch20)
- 频率偏移 : ±175 kHz
- 数据速率 : 500 kbps
- Preamble : 0xAA * 4 bytes
- Sync Word : 0xF68DF68D (4 bytes)

封包格式如下：

- 封包形态 : Fixed Length
- Frame 结构 : Preamble + Sync Word + Payload
- 最大长度 : 16 bytes
- CRC / PHR : 不启用 (默认值)
- 编码 : 无 (NONE)

3.2.3 RX 事件处理(rfEventHandler)

收到 Radio 事件时触发将判断并处理以下两事件

EVT_RX_FRAME

└ while (有待读封包)

```
wise_radio_get_rx_frame_info() // 取得 buffer 地址 + meta
印出: 长度、buffer 地址、RSSI
dump_buffer()                // hex dump payload
wise_radio_release_rx_frame() // 释放 buffer
```

EVT_RX_ERR

└ wise_radio_release_rx_frame() // 释放无效封包

印出 "invalid frame received"

3.2.4 Shell 指令

在初始设置后, 可以透过 UART Shell 控制芯片, 如指定 RF 频道 (如上一节所说 915 MHz~935 MHz 中的其中一个频道), 指令摘要如下:

- `recv on / off` - `static int cmd_recv(int argc, char **argv)`

启动或停止 RX, RX 开启后, 有封包进来时由 `rfEventHandler` 自动处理并印出。

```
RADIO> recv on      → wise_radio_rx_start(intf, ch)
```

```
RADIO> recv off     → wise_radio_rx_stop(intf)
```

- `send [count] [interval_ms]` - `static int cmd_send(int argc, char **argv)`

发送固定测试封包, 预设发 1 次, 间隔 100 ms。

测试封包内容 (固定 16 bytes): 0F 16 11 22 33 44 55 66 77 88 99 AA BB CC DD EE

```
RADIO> send          → 发 1 次
```

```
RADIO> send 5         → 发 5 次, 每次间隔 100 ms
```

```
RADIO> send 5 200     → 发 5 次, 每次间隔 200 ms
```

- `channel [num / list]` - `static int cmd_channel(int argc, char **argv)`

查询或切换频道。

```
RADIO> channel          → 显示目前频道及频率
RADIO> channel list     → 列出全部频道及频率
RADIO> channel 5        → 切换至 ch5; 若 RX 启动中, 自动重启 RX
```

3.2.5 Shell 测试 – 启动讯息

代码入口(demo_radio_trx_main.c - main())会在初始阶段调用 print_banner, 因此正常启动时, 可在 UART 上看到类似欢迎讯息如下:

```
=====
ESMT WISE Demo Application: Radio TRX
Built@ Jul 13 2026 14:19:49
WISE SDK Version 4.0009 2427626f
Radio Platform: subg_soc_9006 V1.10.00
Chip Unique: 0000000000000000
=====
```

指令 help 可列出示例代码所有支持指令

```
RADIO> help
Available commands:
channel      - Select radio channel
send         - Send a packet
recv        - Switch receive on/off
reset        - Do chip reset
info         - Display system info
help         - Show available commands
```

3.2.6 Shell 测试 – 启动 Tx/Rx

如上一章节, 通过 help 命令得到示例支持的功能后, 可知道 send 加上数字即可发送一定数量的封包, 而 recv 加上 on 或 off 可开启或关闭 RX。

如下以两块芯片模块对接测试, 分别为装置 A(负责发送)及装置 B(负责接收)。

可以在图上看到装置 B 先输入 recv on 开启 RX, 接着在装置 A 上输入 send 3 便会连续打出 3 笔固定封包, 装置 B 收到后将详细讯息打印出来。

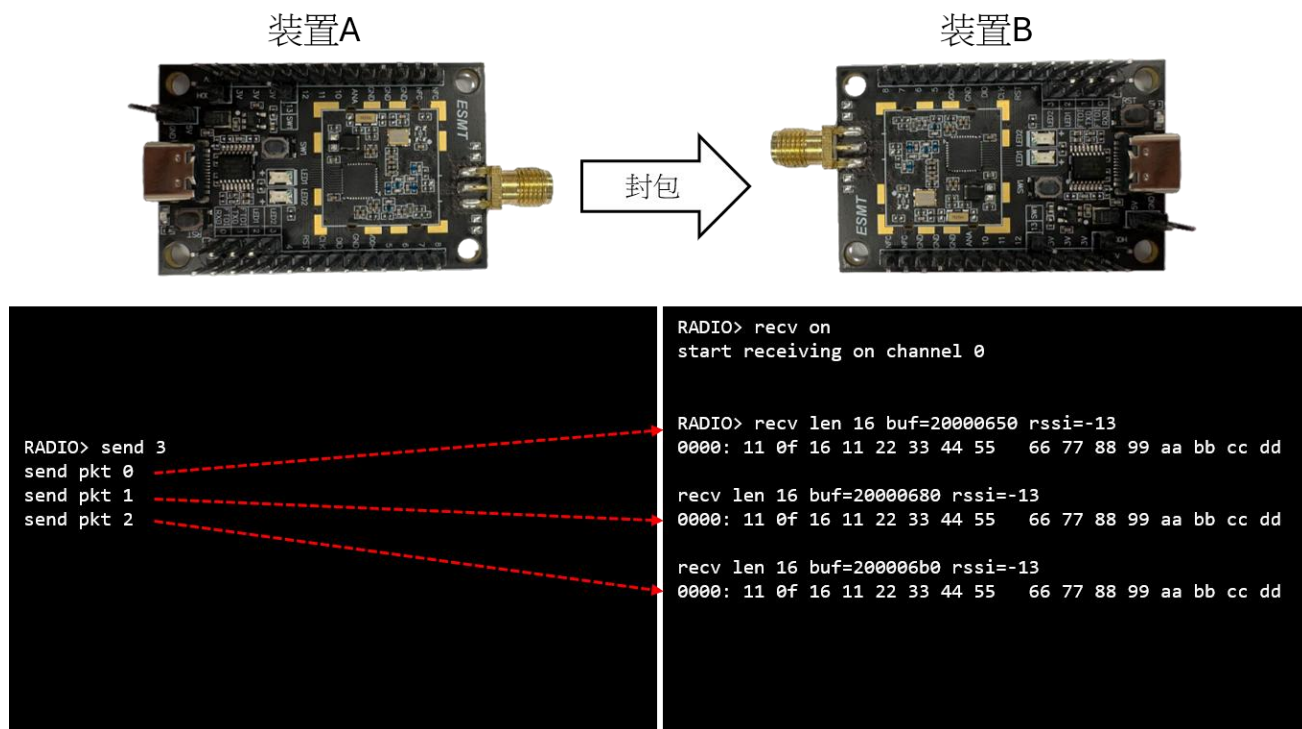


图 11. 芯片模块的对接测试(左:装置 A, 右:装置 B)

4 应用层接口界面

应用层接口界面(APIs)的详细说明可在 WISE_SDK\wise_sdk_release\documents\WISE_API\html\index.html 中开启以下页面:

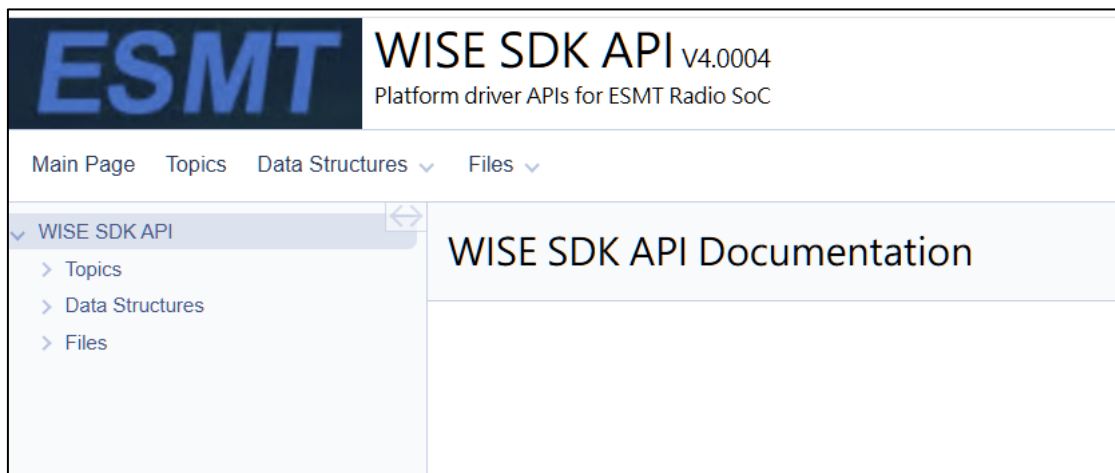


图 12. 应用层接口说明入口

4.1 入口页面摘要

在此页面下可以看到三大分类, Topics(主题), Data Structures(数据结构) 和 Files(档案), 文件将以此三个层次来说明 SDK 的内容。可参考下面三个表格来取得摘要。

表格 7. Topics 页面摘要

分类	说明
Core API	底层硬件抽象与基础服务, 涵盖 GPIO、UART、SPI、I2C、Flash、Radio、PWM、RTC、WDT、PMU、TRNG、EFUSE 等所有周边驱动 API
Middleware	建构在 Core API 之上的可重用软件组件 (如 Flash FileSystem、Shell、System 等中介层)
Protocol	协议堆栈与协议专用工具 (如 WMBus 协定层)
Example APP	范例应用程序与参考实作, 涵盖 AppLoader、GPIO Demo、Radio Shell、FreeRTOS CLI 等各模块的完整示范

表格 8. Data Structure 页面摘要

分类	说明
Data Structures	列出所有有文件说明的结构体 (struct), 附带简短描述, 例如 FLASH_SHAD_RGN_INFO_T、RTC_TIME_T 等
Data Fields	以域名为索引, 列出所有结构体的成员变量, 方便查找特定字段属于哪个结构

表格 9. Files 页面摘要

分类	说明
File List	列出所有有 Doxygen 文件的 .c / .h 原始档，附带档案层级的简述
Globals	汇总所有跨档案的全局符号，包含函式、变量、#define 宏、列举值等，依字母分页索引

4.2 函数摘要

如下再列出 Topics 中的 APIs 及 App 的概括说明如下：

表格 10. Topics 的函数说明摘用

Core API – 底层硬件抽象与基础服务	
函数	说明
WISE Utility APIs	字节序转换辅助、CRC 工具、简易随机数生成及除错辅助函式
WISE Core APIs	核心子系统 API，包含系统初始化、IRQ 控制、临界区段管理及 SDK 版本查询
WISE Crypto APIs	硬件加密加速器设定与数据加解密处理
EFUSE	eFuse 一次性可程序化记忆体操作 API
WISE FIFO APIs	以字节为单位的简易 FIFO 建立与存取
WISE Flash & OTP APIs	Flash/OTP 内存信息查询、读写及抹除操作
WISE GPIO APIs	GPIO 脚位设定、电位控制及中断处理
WISE GPTMR APIs	通用定时器设定、启停控制、事件回呼及计数器工具
WISE I2C APIs	I2C 控制器设定、数据传输及辅助函式
WISE NFC APIs	NFC 设定、内存存取、中断及电源控制
WISE PMU APIs	模块频率闸控 (Clock Gating) 及电源状态查询
WISE PWM APIs	PWM 设定、启停控制及事件回呼注册
WISE Slow PWM APIs	低频 PWM 设定、控制及回呼
WISE Radio APIs	WISE 无线电子系统的公开接口
WISE Radio Wmbus APIs	无线 M-Bus 无线电 PHY 层控制 (Core API)
WISE RTC APIs	实时时钟计时、闹钟设定及事件回呼
WISE SPI APIs	SPI 设定、数据传输及事件回呼接口
SPI Message Format Flags	用于组合 SPI 讯息格式的旗标定义
WISE System APIs	系统初始化、电源控制、DMA 设定、低频振荡器 (LFOSC) 及 ADC
WISE Tick APIs	Tick 计数器及延迟函式工具
WISE TRNG APIs	硬件真随机数生成器 (TRNG) 初始化与随机数取得
WISE UART APIs	UART 设定、数据传输及中断处理

Core API - 底层硬件抽象与基础服务	
函数	说明
WISE Watchdog Timer APIs	看门狗定时器设定、控制、逾时处理及回呼
WISE Wake-Up Timer APIs	单次 / 周期性唤醒定时器操作及回呼
Example APP - 范例应用程序与参考实作	
函数	说明
WISE Utility APIs	字节序转换辅助、CRC 工具、简易随机数生成及除错辅助函数
WISE Core APIs	核心子系统 API, 包含系统初始化、IRQ 控制、临界区段管理及 SDK 版本查询
WISE Crypto APIs	硬件加密加速器设定与数据加解密处理
EFUSE	eFuse 一次性可程序化记忆体操作 API
WISE FIFO APIs	以字节为单位的简易 FIFO 建立与存取
WISE Flash & OTP APIs	Flash/OTP 内存信息查询、读写及抹除操作
WISE GPIO APIs	GPIO 脚位设定、电位控制及中断处理
WISE GPTMR APIs	通用定时器设定、启停控制、事件回呼及计数器工具
WISE I2C APIs	I2C 控制器设定、数据传输及辅助函数
WISE NFC APIs	NFC 设定、内存存取、中断及电源控制
WISE PMU APIs	模块频率闸控 (Clock Gating) 及电源状态查询
WISE PWM APIs	PWM 设定、启停控制及事件回呼注册
WISE Slow PWM APIs	低频 PWM 设定、控制及回呼
WISE Radio APIs	WISE 无线电子系统的公开接口
WISE Radio Wmbus APIs	无线 M-Bus 无线电 PHY 层控制 (Core API)
WISE RTC APIs	实时时钟计时、闹钟设定及事件回呼
WISE SPI APIs	SPI 设定、数据传输及事件回呼接口
SPI Message Format Flags	用于组合 SPI 讯息格式的旗标定义
WISE System APIs	系统初始化、电源控制、DMA 设定、低频振荡器 (LFOSC) 及 ADC
WISE Tick APIs	Tick 计数器及延迟函数工具
WISE TRNG APIs	硬件真随机数生成器 (TRNG) 初始化与随机数取得
WISE UART APIs	UART 设定、数据传输及中断处理
WISE Watchdog Timer APIs	看门狗定时器设定、控制、逾时处理及回呼
WISE Wake-Up Timer APIs	单次 / 周期性唤醒定时器操作及回呼